

Network

Network: A network in this context refers to a collection of devices (such as computers, servers, printers, etc.) that are connected together and can communicate with each other using a common set of rules or protocols. In the context of IP addresses, a network typically represents a portion of the Internet or an isolated network within an organization. IP addresses are assigned to devices within a network to facilitate communication between them.

Class: The concept of IP address classes is an older system for classifying IP addresses. It's not as commonly used today due to the adoption of Classless Inter-Domain Routing (CIDR), but it's still worth understanding:

Class A: Class A IP addresses start with a number from 1 to 126 in the first octet (the first 8 bits). They are typically used for very large networks, with the first octet representing the network portion and the remaining three octets for hosts. For example, 10.0.0.0 is a Class A network.

Class B: Class B IP addresses start with a number from 128 to 191 in the first octet. They are commonly used for medium-sized networks, with the first two octets representing the network portion and the remaining two for hosts. For example, 172.16.0.0 is a Class B network.

Class C: Class C IP addresses start with a number from 192 to 223 in the first octet. They are typically used for smaller networks, with the first three octets representing the network portion and the last octet for hosts. For example, 192.168.0.0 is a Class C network.

Class D and E: These classes are reserved for special purposes. Class D is used for multicast groups, and Class E is reserved for experimental purposes.

Class A:

IP addresses in Class A networks start with a number from 1 to 126 in the first octet. The default subnet mask for Class A is 255.0.0.0, which means the first octet represents the network portion, and the remaining three octets are for host addresses. The range of Class A IP addresses is from 1.0.0.0 to 126.255.255.255.

Class B:

IP addresses in Class B networks start with a number from 128 to 191 in the first octet. The default subnet mask for Class B is 255.255.0.0, which means the first two octets represent the network portion, and the remaining two octets are for host addresses. The range of Class B IP addresses is from 128.0.0.0 to 191.255.255.255.

Class C:

IP addresses in Class C networks start with a number from 192 to 223 in the first octet. The default subnet mask for Class C is 255.255.255.0, which means the first three octets represent the network portion, and the last octet is for host addresses. The range of Class C IP addresses is from 192.0.0.0 to 223.255.255.255.

Class D:

Class D IP addresses are reserved for multicast groups and not typically used for host addressing. The range of Class D IP addresses is from 224.0.0.0 to 239.255.255.255.

Class E:

Class E IP addresses are reserved for experimental purposes and should not be used for general networking.

The range of Class E IP addresses is from 240.0.0.0 to 255.255.255.255.

Determine the Network Address:

Start with the IP address and subnet mask in CIDR notation (e.g., 192.168.1.0/24).

Convert the subnet mask to binary form. In this case, a /24 subnet mask means the first 24 bits are for the network portion, and the remaining 8 bits are for host addresses, so the binary representation of the subnet mask is 11111111.11111111.11111111.00000000.

Apply bitwise AND operation between the IP address and the subnet mask to find the network address:

IP Address: 11000000.10101000.00000001.00000000 (192.168.1.0 in binary)

Subnet Mask: 11111111.11111111.11111111.00000000 (/24 in binary)

Network Address: 11000000.10101000.00000001.00000000

Calculate the Number of Hosts:

Determine how many bits are available for host addresses based on the subnet mask. In this case, there are 8 bits for host addresses (/24 means 8 bits for hosts).

Calculate the number of possible host addresses using 2^n , where n is the number of host bits. So, $2^8 = 256$ possible host addresses.

Identify the Range:

The range of host addresses within the subnet is from 0 to 255 (since there are 256 possible addresses).

The first host address is the network address + 1 (192.168.1.1 in this case), and the last host address is the network address + the number of possible hosts - 1 (192.168.1.254 in this case).

So, for the subnet 192.168.1.0/24:

Network Address: 192.168.1.0

First Host Address: 192.168.1.1

Last Host Address: 192.168.1.254

Broadcast Address: 192.168.1.255 (reserved for broadcasting to all hosts in the subnet)

Number of Hosts: 256 (0 to 255)

Amazon VPC

Amazon VPC: Amazon VPC is the fundamental networking service in AWS. It allows you to provision a logically isolated section of the AWS cloud where you can launch AWS resources, such as EC2 instances (virtual servers), RDS databases, and more, into a virtual network that you define.

Subnets: Within an Amazon VPC, you can create multiple subnets. Subnets are smaller address ranges within your VPC. You can think of them as segmented networks within your virtual network. Subnets can be either public (accessible from the internet) or private (not directly accessible from the internet).

IP Addressing: When you create an Amazon VPC, you specify an IP address range for it, typically using CIDR notation. This defines the overall address space for your VPC. Subnets within the VPC are assigned IP address ranges that fall within the VPC's address space.

Routing: Amazon VPC allows you to set up routing tables to control how traffic flows between different subnets and between your VPC and the internet. You can configure Network Address Translation (NAT) gateways and internet gateways to control the flow of traffic to and from your VPC.

Security Groups and Network Access Control Lists (NACLs): These are used to control inbound and outbound traffic at the instance level and subnet level, respectively, in your VPC. Security groups act as firewalls at the instance level, while NACLs act at the subnet level.

Peering and VPN: Amazon VPCs can be connected to each other using VPC peering, allowing them to communicate securely over the AWS network. You can also set up Virtual Private Network (VPN) connections to connect your VPC to on-premises networks.

Direct Connect: AWS Direct Connect is a service that allows you to establish a dedicated network connection from your on-premises data center to AWS, bypassing the public internet for enhanced security and reliability.

Transit Gateway: AWS Transit Gateway simplifies network architecture by allowing you to connect multiple VPCs, VPNs, and Direct Connect gateways in a hub-and-spoke model.

VPC Endpoints: These are used to privately connect your VPC to AWS services (e.g., S3, DynamoDB) without going over the internet.

Creating a Virtual Private Cloud (VPC)

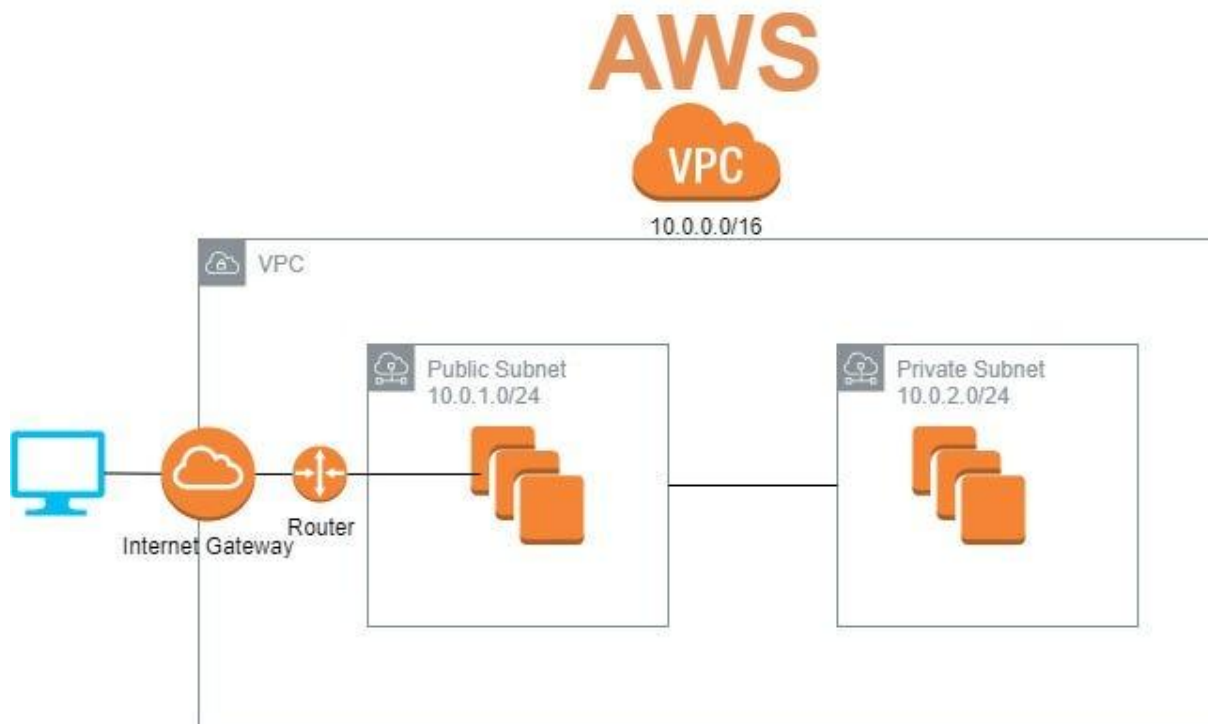
Sign in to the AWS Management Console:

Go to the AWS Management Console at <https://aws.amazon.com/>.

Sign in using your AWS account credentials.

Navigate to the VPC Dashboard

Once logged in, go to the AWS Management Console.



In the top-right corner, click on "Services" and then select "VPC" under the "Networking & Content Delivery" section. This will take you to the VPC Dashboard.

Create a VPC:

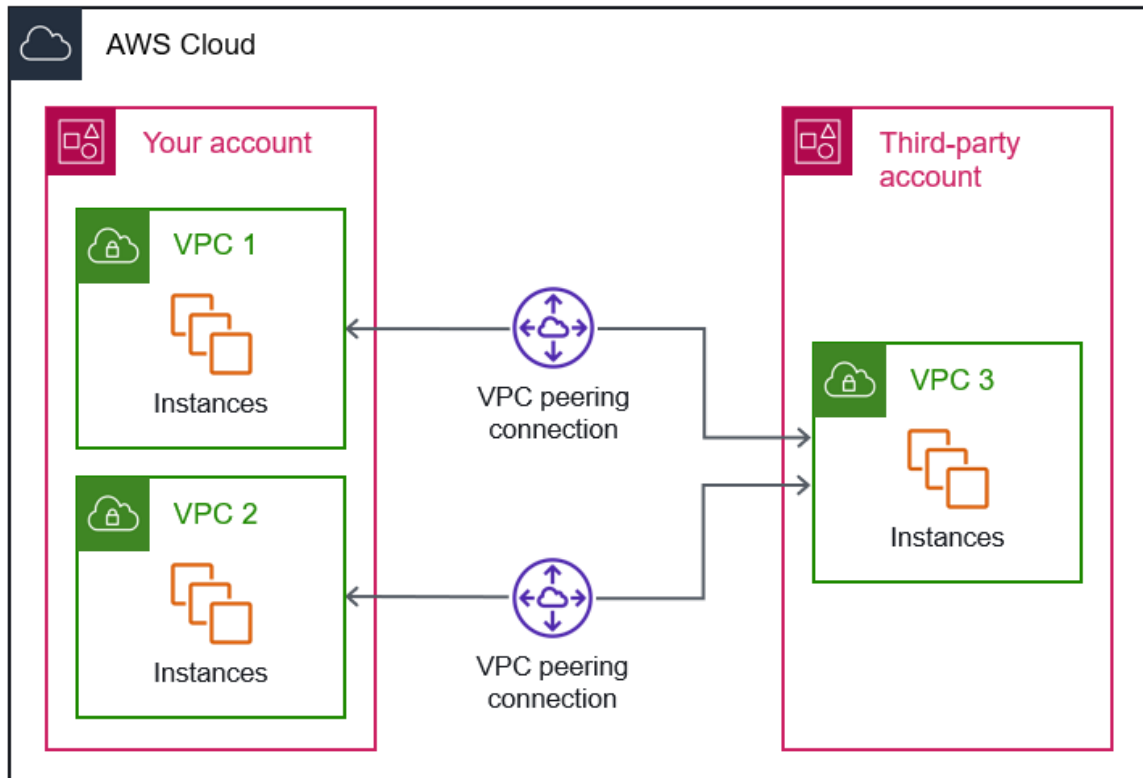
In the VPC Dashboard, click on the "Create VPC" button.

Configure VPC Settings:

1. Give your VPC a name and provide an IPv4 CIDR block. This block defines the IP address range for your VPC. For example, you can use 10.0.0.0/16 for a large VPC.
2. Configure Additional Options (Optional):
3. You can configure DHCP options, enable DNS support, and add IPv6 CIDR blocks if needed. These settings depend on your specific requirements.
4. Create Subnets:
5. After creating the VPC, you should create one or more subnets within it. Subnets are used to group resources in different availability zones within the VPC.
6. Go to the "Subnets" section in the VPC Dashboard and click "Create Subnet."
7. Provide a name, select the VPC you created earlier, choose an availability zone, and specify a CIDR block for the subnet.
8. Configure Route Tables:
9. Route tables define how traffic is routed within your VPC. By default, AWS creates a main route table for your VPC.

10. You can create custom route tables to control traffic routing for specific subnets. Go to the "Route Tables" section and click "Create Route Table."
11. Attach Subnets to Route Tables:
12. After creating route tables, attach them to the subnets. This determines the routing behaviour for traffic in those subnets.
13. Configure Security Groups and Network ACLs:
14. Security groups and Network ACLs are used to control inbound and outbound traffic to your instances. You can configure these settings under the "Security Groups" and "Network ACLs" sections in the VPC Dashboard.
15. Create and Associate an Internet Gateway (Optional):
16. If you want resources within your VPC to have internet access, create an Internet Gateway and attach it to your VPC.
17. Launch EC2 Instances or Other Resources:
18. Once your VPC is set up, you can launch AWS resources such as EC2 instances, RDS databases, and more into your VPC.

VPC Peering



1. VPC Peering is a networking connection that allows you to connect one VPC with another VPC through a direct network
2. You can peer VPC's with one AWS accounts and other accounts VPCs
You can peer between regions.
3. you have one VPC in one region and other VPC in another region, then you can peer the VPCs between different regions.

How connected aws one account to another account --> ex: a & B

under Select another VPC to peer with we're going to provide the details of the VPC in the other AWS account. Select another account and enter the account B account id. if your other account .
different region, under Region select Another Region and choose the region for VPC ID (Accepter) enter the VPC id of the VPC in account

Setting Up VPC Peering:

1. Configuring Route Tables:

Update the route tables in the VPCs to include routes for the IP address range of the peered VPC.

2. Authorization:

Both VPCs involved in peering must explicitly authorize the connection. This involves accepting a peering request initiated by the owner of the requesting VPC.

3. DNS Resolution:

Ensure that DNS resolution is enabled between the peered VPCs to allow seamless communication using private DNS hostnames.

4. Security Groups and Network ACLs:

Configure security groups and network ACLs to control inbound and outbound traffic between the peered VPCs, ensuring a secure environment.

Benefits of VPC Peering:**1. Simplified Network Architecture:**

VPC Peering simplifies the network design by allowing multiple VPCs to communicate directly without the need for a complex mesh of VPN connections or physical hardware.

2. Cost-Effective Data Transfer:

Since VPC Peering utilizes the AWS backbone network, data transfer between peered VPCs is cost-effective and does not incur additional data transfer charges.

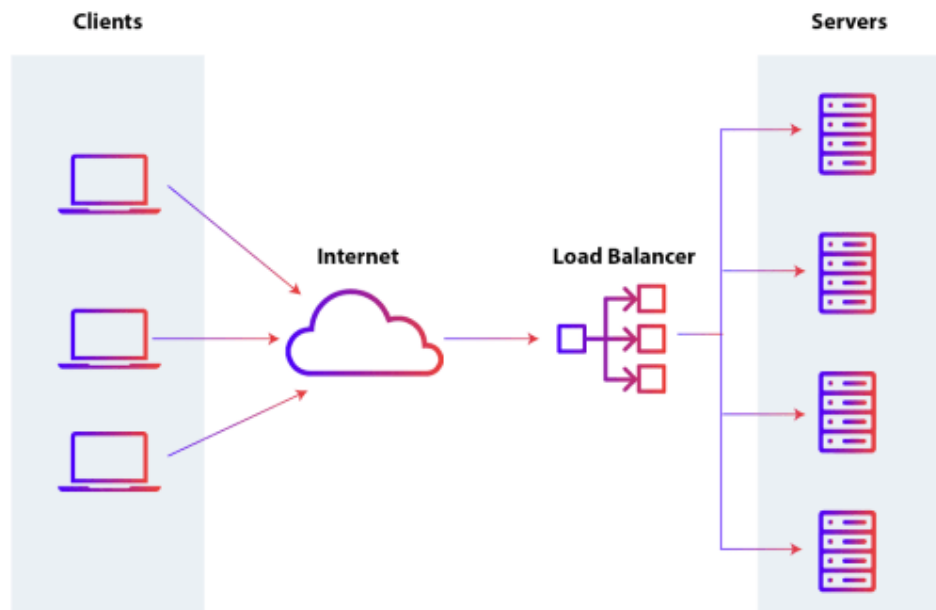
3. Resource Sharing:

VPC Peering facilitates resource sharing between different VPCs. This includes sharing of Amazon EC2 instances, databases, and other services without exposing them to the public internet.

4. Security and Isolation:

Communication between VPCs through peering remains private and does not traverse the public internet. Network ACLs and security groups can be configured to control traffic between peered VPCs

LOAD BALANCING



Load balancing in AWS refers to the distribution of incoming network traffic across multiple Amazon Elastic Compute Cloud (Amazon EC2) instances or other AWS resources in order to ensure high availability and fault tolerance of your applications. AWS provides a service called Elastic Load Balancing (ELB) to achieve this.

Elastic Load Balancing offers three types of load balancers:

Application Load Balancer (ALB): ALB is best suited for routing HTTP/HTTPS traffic and is capable of advanced request routing based on content and context, making it ideal for web applications. It supports path-based routing, host-based routing, and routing based on request headers.

Network Load Balancer (NLB): NLB is designed for handling TCP and UDP traffic and is used when you need extremely high performance and low-latency capabilities. It is often used for non-HTTP/TCP-based services like gaming, IoT, and other custom protocols.

Classic Load Balancer (CLB): CLB is the original load balancer service in AWS and is suitable for applications that require basic load balancing of HTTP/HTTPS traffic. AWS recommends using ALB or NLB for new applications.

Here are the steps to create a load balancer using Elastic Load Balancing in AWS:

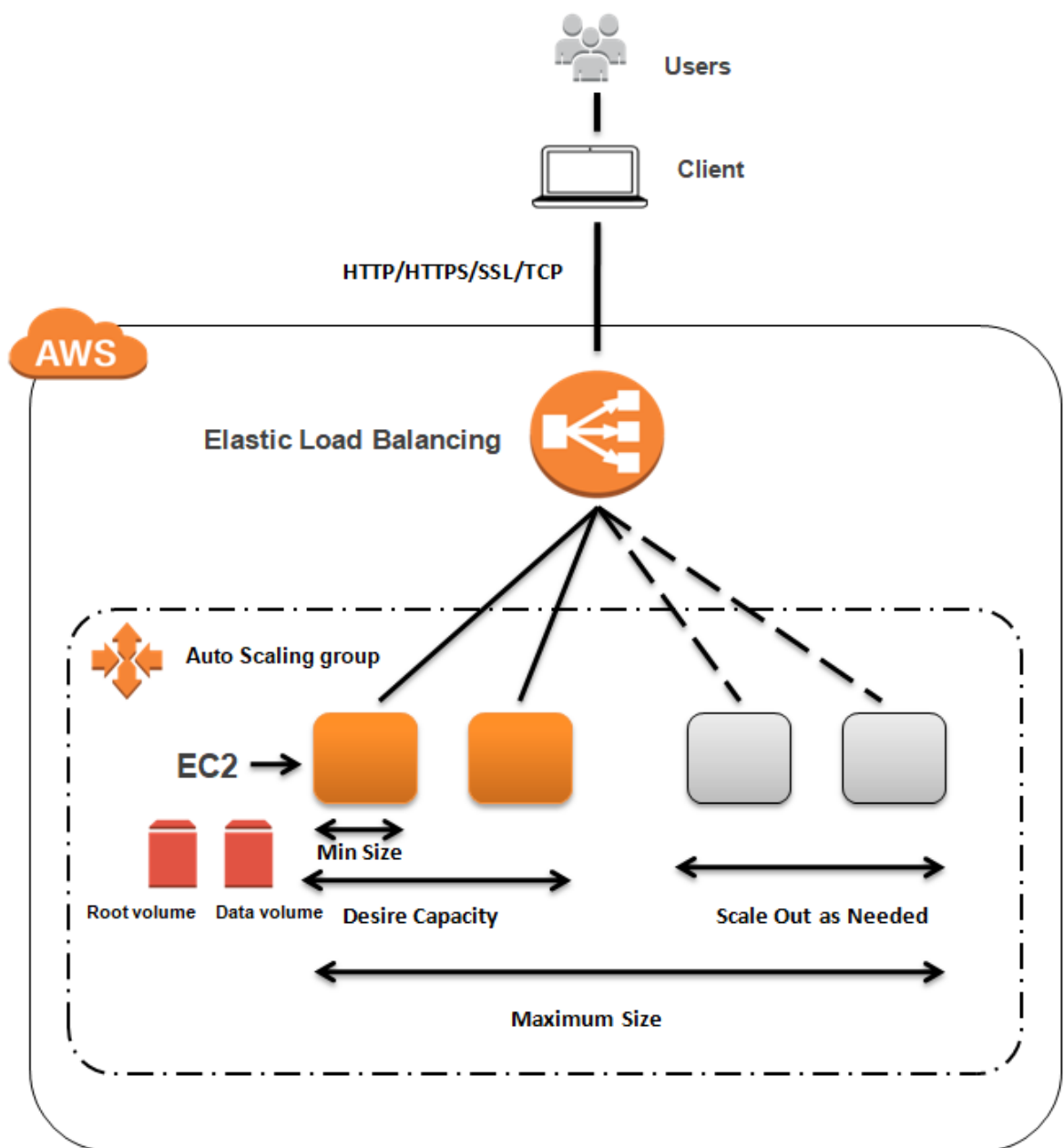
1. Sign in to AWS Console: Log in to your AWS Management Console.
2. Navigate to Load Balancers: Go to the Elastic Load Balancing (ELB) section. You can find it in the "Services" dropdown or by searching for "Load Balancers" in the AWS Console.

3. Choose Load Balancer Type: Click on the "Create Load Balancer" button and choose the type of load balancer you want to create (Application Load Balancer, Network Load Balancer, or Classic Load Balancer).
4. Configure Load Balancer: Fill out the necessary configuration details for your load balancer, including its name, listener ports, security groups, and subnet settings. The specific configuration options will vary depending on the type of load balancer you're creating.
5. Define Target Group (for ALB and NLB): For Application Load Balancers (ALB) and Network Load Balancers (NLB), you need to define a target group. A target group specifies the targets (such as EC2 instances) that will receive traffic from the load balancer. You'll need to specify the target type, protocol, and port for the target group.
6. Add Instances (Targets): Associate the target group with the instances (or other resources) that should receive traffic from the load balancer. You can specify instances manually or use tags to include instances dynamically.
7. Configure Health Checks: Define health checks that the load balancer will perform on the target instances to determine their health. The load balancer will automatically route traffic only to healthy instances.
8. Add Tags (Optional): You can add tags to your load balancer for easier management and resource identification.
9. Review and Create: Review all your settings and configurations, and if everything looks correct, click the "Create" button to create the load balancer.
10. Wait for Provisioning: AWS will provision your load balancer, and you can monitor its status in the AWS Management Console. Once it's in the "Active" state, it's ready to start routing traffic.
11. Update DNS: If your application uses a custom domain, you may need to update your DNS records to point to the DNS name associated with your load balancer.

Auto Scaling

Auto Scaling

Auto Scaling in AWS is a service that allows you to automatically adjust the number of resources in your AWS infrastructure based on demand. **It helps maintain the desired number of instances, ensuring that your application can handle varying workloads efficiently without over-provisioning or under-provisioning resources.**



2 TYPES OF AUTO SCALING

1. Horizontal Auto scaling

2. Vertical Auto Scaling

Horizontal Auto Scaling

Horizontal Auto scaling Also known as dynamic scaling, horizontal auto scaling involves adding or removing instances to match the current workload demand. This type of auto scaling adjusts the number of instances within an Auto Scaling Group (ASG) based on predefined policies or conditions. Horizontal scaling is commonly used to handle fluctuations in traffic or workload, ensuring that the application can efficiently manage varying levels of demand without being over-provisioned or under-provisioned.

Scaling Policies: Horizontal auto scaling policies define when and how to add or remove instances based on metrics such as CPU utilization, network traffic, or custom application metrics. These policies can be configured to scale out (add instances) or scale in (remove instances) dynamically in response to changes in workload.

Vertical Auto Scaling

Vertical auto scaling involves adjusting the size or capacity of individual instances to meet changing workload demands. Unlike horizontal scaling, which adds or removes instances, vertical scaling modifies the resources (such as CPU, memory, or storage) of existing instances to handle increased or decreased load.

Instance Type Changes: In vertical auto scaling, you may upgrade or downgrade the instance type to increase or decrease the available resources. For example, you could switch from a smaller instance type to a larger one with more CPU and memory capacity to handle increased demand.

Elastic Load Balancing (ELB) Integration: Vertical auto scaling can also work in conjunction with Elastic Load Balancing (ELB) to distribute traffic across instances efficiently. When using ELB, vertical scaling can adjust the capacity of instances behind the load balancer dynamically to maintain optimal performance.

Components of AWS Auto Scaling

Auto Scaling Groups (ASGs): These are the core component of Auto Scaling. ASGs define a collection of EC2 instances (or other scalable resources) that is treated as a logical unit for scaling and management. You specify the minimum, maximum, and desired number of instances in an ASG.

Launch Configuration or Launch Template: You define the configuration details for the instances launched by Auto Scaling using either a launch configuration or a launch template. This includes specifications such as the Amazon Machine Image (AMI), instance type, key pair, security groups, and block device mapping.

Scaling Policies: You define scaling policies to instruct Auto Scaling on how to adjust the number of instances in response to changing demand. Scaling policies can be based on metrics such as CPU utilization, network traffic, or custom metrics.

Dynamic Scaling: Auto Scaling can scale your resources automatically in response to changing demand, based on predefined scaling policies. This ensures that your application always has the appropriate amount of resources to handle traffic fluctuations.

Scheduled Scaling: You can also set up scheduled scaling actions to increase or decrease the number of instances at specific times. This can be useful for anticipated changes in demand, such as daily or weekly spikes.

Health Checks: Auto Scaling automatically replaces instances that fail health checks, ensuring the overall health and availability of your application.

Auto Scaling Policy Components:

Adjustment Type:

Specifies how the capacity of the Auto Scaling Group should be adjusted. Common adjustment types include:

Change In capacity: Increase or decrease the current capacity by a fixed number.

Percent Change In Capacity: Increase or decrease the current capacity by a percentage.

Scaling Adjustment:

The number of instances or percentage by which to adjust the current capacity. This value can be positive or negative, depending on whether you want to scale up or down.

Cooldown:

Cooldown periods prevent Auto Scaling from launching or terminating additional instances before the previous scaling activity takes effect. This helps to prevent rapid fluctuations due to brief traffic spikes.

Types of Auto Scaling Policies:

Simple Scaling Policy:

This type of policy adjusts the capacity of your Auto Scaling group based on a single, specified Cloud Watch metric. For example, you could scale based on the average CPU utilization of instances.

Step Scaling Policy:

This policy adjusts the capacity based on the size of the alarm breach. It allows you to scale the number of instances in your Auto Scaling group in multiple steps based on Cloud Watch alarms.

Target Tracking Scaling Policy:

This policy automatically adjusts the capacity of your Auto Scaling group to maintain a specified target value for a chosen metric, such as average CPU utilization or request count.

Steps by step guide for setup AWS Auto Scaling

1. Login to your AWS Console and navigate to the EC2 service.
2. Click on the "Auto Scaling Groups" option from the left-hand menu.
3. Click on the "Create Auto Scaling Group" button to start the auto-scaling group creation process.
4. Give an auto scaling group name and click on the "Create a launch template").
5. Give a name and description for launch template.
6. Configure AMI for your application (Webserver).
7. Configure instance type as per your need.
8. Configure Key pair, Security group and Network settings.
9. Configure [EBS storage](#) volumes and click on the "Create launch template".

A launch template will be created.

10. Now, Go back to the screen "Create Auto scaling group". Click on the refresh icon to get created launch configuration and select that launch template. After configuration click on the "Next"
11. Choose the Network and subnet(s) configuration where you want to launch the EC2 instances.
You can select multiple subnets to ensure high availability.
After configuration click on the "Next"
12. Optional step: Create or attach a [Load balancer](#) or target group to distribute traffic across instances.
13. Setup the minimum, maximum, and desired number of EC2 instances that should be running at any time.
14. You can define scaling policies if required. Target tracking scaling adjusts the number of instances based on a predefined metric, while step scaling adds or removes instances based on thresholds that you define.
For example, Here for metric type "Average CPU utilization" the target value is 50 defined. After configuration click on the "Next"
15. Optional step: Add notification, Add Tags (skipping for this lab). Click on the "Next"
16. Review your all configuration and click on the "Create Auto Scaling group" button
17. After you have created your auto-scaling group, AWS will automatically launch desired EC2 instances based on your defined scaling policies. You need to wait for some time and refresh your EC2 instance dashboard.
18. To validate the AutoScaling configuration, let's terminate one instance after sometime (using health check configuration), AutoScaling will automatically create a new instance to meet your configured desired capacity of instances.

Note: Make sure to delete all the AWS resources created for this lab to avoid unnecessary charges.

